Organization of Digital Computer Lab
EECS112L

Lab #: 4
Student Name:
VATSA GUPTA
Student ID#:
88321416
Date: March 1 2026

1 Objective

The objective of this lab is to design, implement, and verify a 32-bit 5-stage pipelined MIPS processor using Verilog. The processor must correctly execute a sequence of instructions while improving performance through instruction pipelining. In addition, the lab aims to demonstrate and analyze pipeline hazards and the hardware techniques used to resolve them.

Specifically, the goals are:

1. To implement the five pipeline stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory (MEM), and Write Back (WB).

2. To implement pipeline registers between stages to allow simultaneous execution of multiple instructions.

3. To implement a forwarding (bypassing) unit to resolve data dependencies between consecutive instructions.

4. To implement a hazard detection unit that inserts stalls for load-use data hazards.

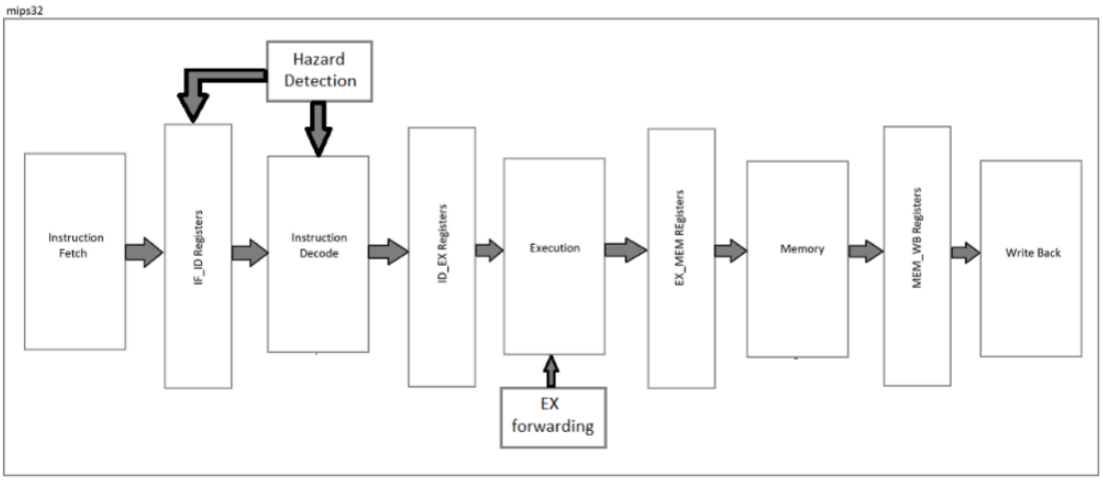
5. To implement control hazard handling by flushing the pipeline during branch and jump instructions.

6. To verify correct processor operation using simulation waveforms and memory output results.

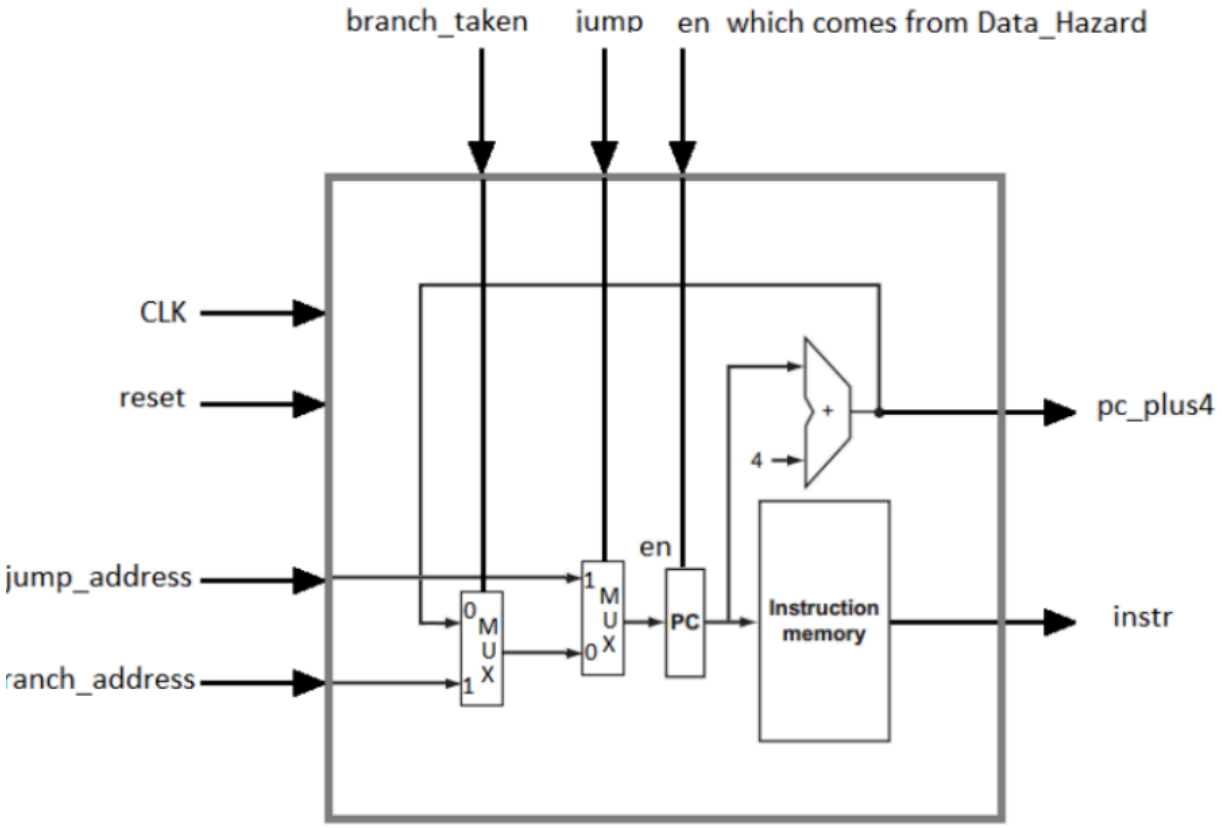
The successful completion of the lab demonstrates correct pipelined execution, proper hazard handling, and accurate instruction execution as verified through expected data memory value

The following is the figure for a pipelined processor:-

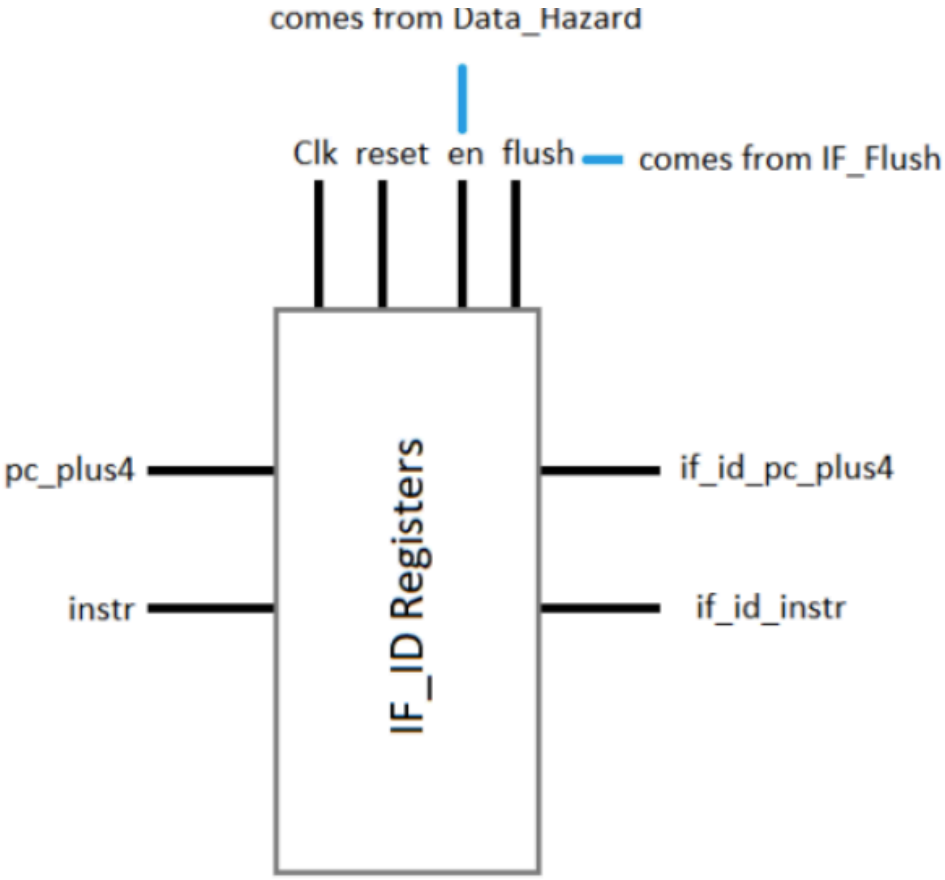
Figure 1: Pipeline processor



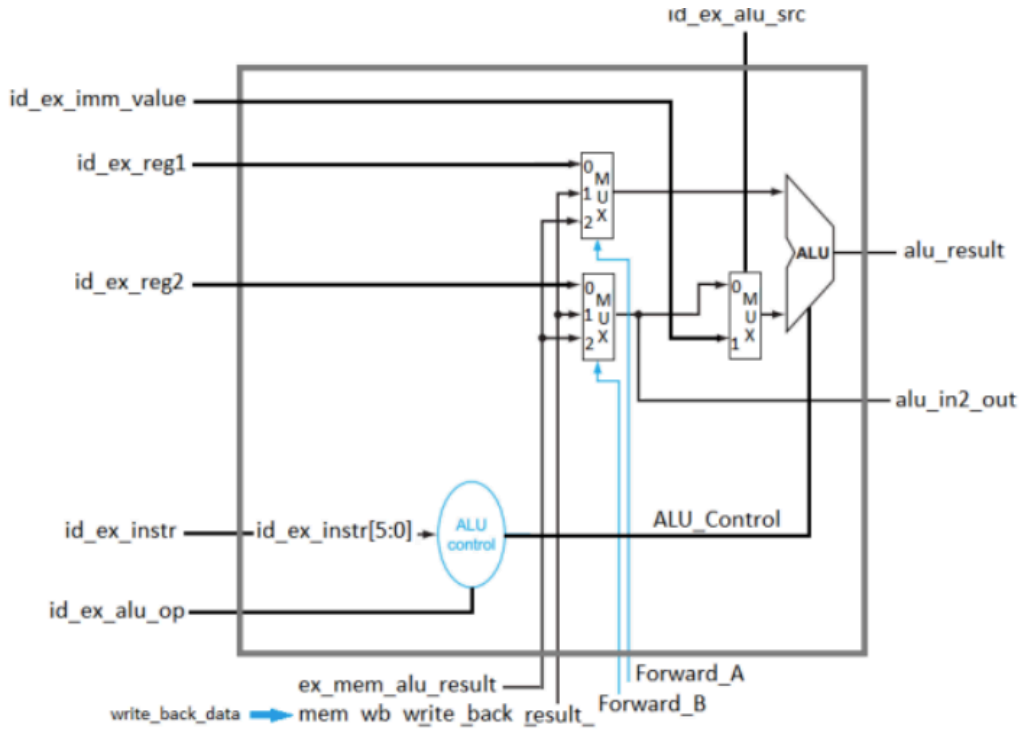
Various block diagrams are brought together to complete the entire mips processor. Here are some examples of our block diagrams:-


branch_taken
jump
en
which comes from Data_Hazard
CLK
reset
jump_address
ranch_address
0
M
U
X
1
1
M
U
X
0
en
PC
+
4
Instruction
memory
pc_plus4
instr

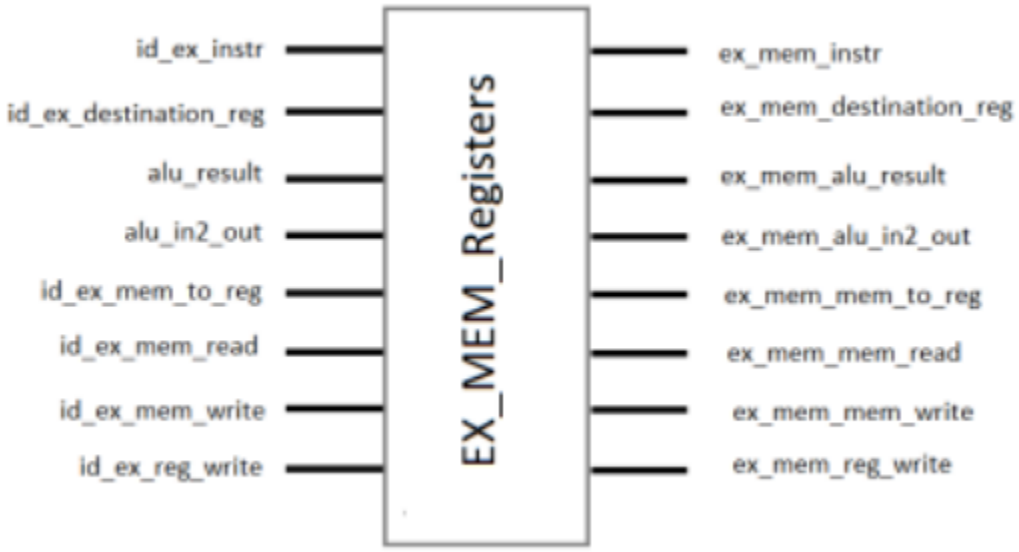
IF/ID Stage:-



EX Stage/Forwarding unit:-



Mem /Ex stage:-



Other blocks are present too.

2. Procedure

1. **Design Implementation**
   A 5-stage pipelined 32-bit MIPS processor was implemented in Verilog. The processor consists of the following pipeline stages:

   - Instruction Fetch (IF)
   - Instruction Decode/Register Read (ID)
   - Execute (EX)
   - Memory Access (MEM)
   - Write Back (WB)

Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) were added between stages to allow multiple instructions to be processed simultaneously.

---

2. **Instruction and Data Memory Initialization**
   The instruction memory (ROM) was initialized with a program containing:

   - arithmetic and logical instructions
   - shift operations
   - load and store instructions
   - multiply and divide instructions
   - branch and jump instructions

The data memory (RAM) was preloaded with known hexadecimal values in locations 0–9 using the testbench so that correct results could be verified after execution.

---

3. **Hazard Handling Hardware**

The following hazard handling mechanisms were implemented:

a) Data Hazard Detection (Load-Use Hazard)

- A hazard detection unit monitors the instruction in the ID stage and the load instruction in the EX stage.
- If the destination register of a lw instruction matches a source register of the next instruction, a stall is inserted.
- During the stall:
  - PC is frozen
  - IF/ID register is frozen
  - ID/EX control signals are cleared to insert a NOP

b) Forwarding (Bypassing)

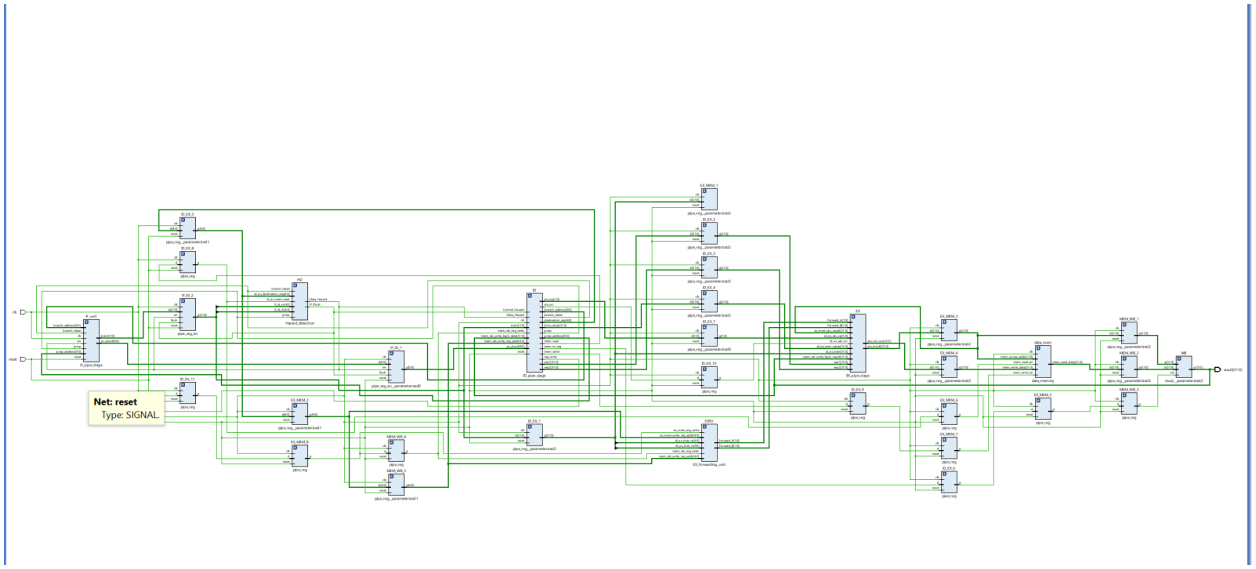
- A forwarding unit was implemented to eliminate unnecessary stalls.
- Results from EX/MEM and MEM/WB stages are forwarded directly to the ALU inputs.
- This allows dependent arithmetic instructions to execute without waiting for register write-back.

c) Control Hazard Handling

- Branch and jump instructions are resolved in the Decode stage.
- When a branch is taken or a jump occurs:
  - PC is redirected to the target address
  - IF/ID pipeline register is flushed
  - A NOP is inserted to remove the wrongly fetched instruction

4. **Schematic:-**

Here are the schematics for entire processor:-



mips_32.v

3. Simulation Results

No Dependancy:-

In this section, instructions do not immediately depend on the previous instruction's output. Therefore, when each instruction reaches the ID stage, the correct operands are already available in the register file. The pipeline proceeds normally without stalls or forwarding.

Memory[11] — ANDI

Instruction: andi r11, r3, 0xFF63
Operands: r3 = 0x0FD76E10
Operation: 0x0FD76E10 AND 0xFFFF_FF63
Result: 0x0FD76E00
Explanation: The immediate masks the low bits of r3. Value read directly from register file.

Memory[12] — NOR

Instruction: nor r12, r1, r2
Operands: r1 = 0x00000001, r2 = 0x0FD76E10
Operation: ~(r1 | r2)
Result: 0xF02891EE
Explanation: Both operands already available; ALU performs bitwise NOR.

Memory[13] — SLT

Instruction: slt r13, r1, r2
Operands: r1 = 1, r2 = 0x0FD76E10
Operation: r1 < r2 → true
Result: 0x00000001

Memory[14] — SLL

Instruction: sll r14, r2, 3
Operands: r2 = 0x0FD76E10
Operation: r2 << 3
Result: 0x7EBB7080

Memory[15] — SRL

Instruction: srl r15, r1, 5
Operands: r1 = 0x00000001
Operation: logical shift right 5
Result: 0x00000000

Memory[16] — SRA

Instruction: sra r16, r6, 6
Operands: r6 = 0x80000000
Operation: arithmetic right shift 6
Result: 0xFE000000

Memory[17] — XOR

Instruction: xor r17, r2, r3
Operands: r2 = 0x0FD76E10, r3 = 0x0FD76E10
Operation: XOR identical values
Result: 0x00000000

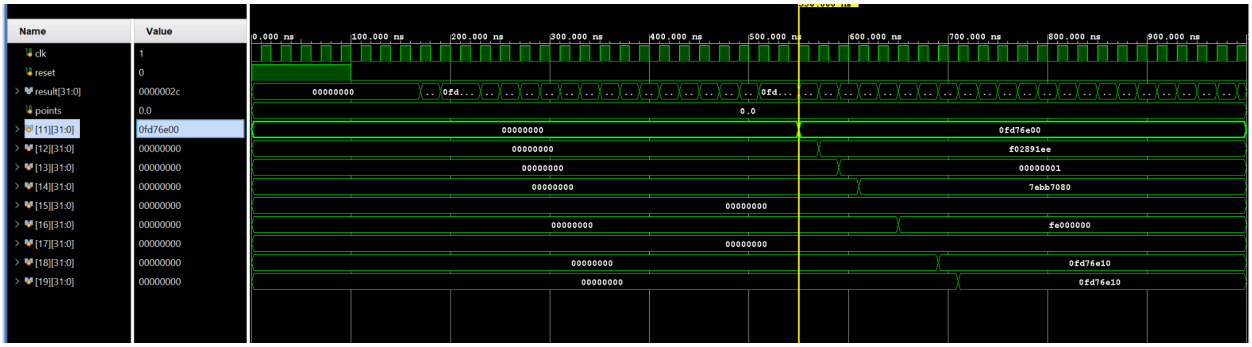
Memory[18] — MULT

Instruction: mult r18, r1, r2
Operands: r1 = 1, r2 = 0x0FD76E10
Operation: multiply
Result: 0x0FD76E10

Memory[19] — DIV

Instruction: div r19, r2, r1
Operands: r2 = 0x0FD76E10, r1 = 1
Operation: divide
Result: 0x0FD76E10

The following is the waveform for this:-



Please zoom in for a full view.

Conclusion:
All results are correct with no pipeline hazards, proving normal ALU and register write-back operation.

Memory Locations 20–28 : Forwarding (Handling RAW Hazards)

Memory[20] — ANDI

Instruction: andi r11, r7, 0x0F63
Operands: r7 = 0x9012FD65
Operation: 0x9012FD65 & 0x00000F63
Result: 0x00000D61
(No hazard)

Memory[21] — NOR (Forward EX/MEM → EX)

Instruction: nor r12, r2, r11
Operands: r2 = 0x0FD76E10, r11 = 0x00000D61
Operation: ~(r2 | r11)
Result: 0xF028908E
Explanation: r11 not yet written back; forwarded from EX/MEM pipeline register.

Memory[22] — SLT (Forward MEM/WB → EX)

Instruction: slt r13, r11, r2
Operands: r11 = 0x00000D61, r2 = 0x0FD76E10
Operation: r11 < r2 → true
Result: 0x00000001
Explanation: r11 forwarded from MEM/WB stage.

Memory[23] — SLL

Instruction: sll r14, r7, 13
Operands: r7 = 0x9012FD65
Operation: shift left
Result: 0x5FACA000

Memory[24] — SRL (Forward EX/MEM → EX)

Instruction: srl r15, r14, 7
Operands: r14 = 0x5FACA000
Operation: shift right
Result: 0x00BF5940

Explanation: r14 forwarded directly from previous instruction.

Memory[25] — SRA (Forward MEM/WB → EX)

Instruction: sra r16, r14, 2
Operands: r14 = 0x5FACA000
Operation: arithmetic shift
Result: 0x17EB2800

Memory[26] — XOR

Instruction: xor r17, r2, r7
Operands: r2 = 0x0FD76E10, r7 = 0x9012FD65
Result: 0x9FC59375

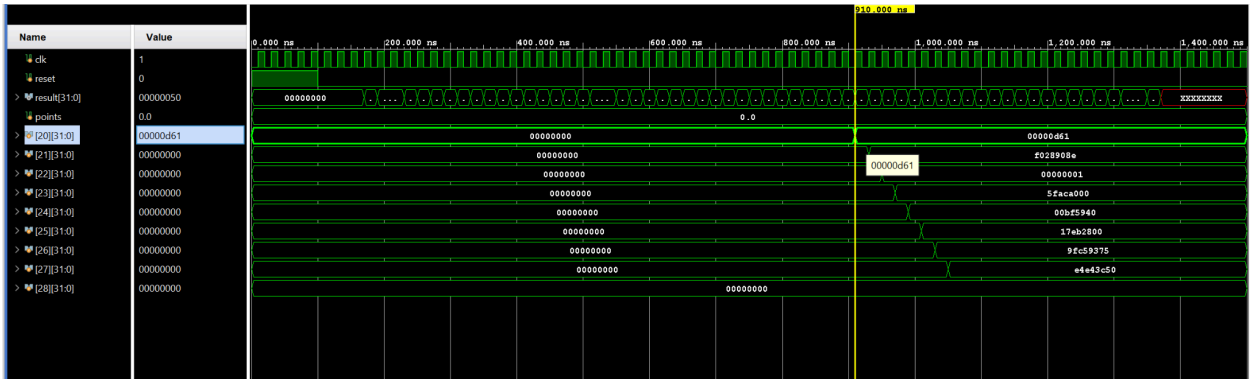
Memory[27] — MULT

Instruction: mult r18, r2, r7
Operands: r2 = 0x0FD76E10, r7 = 0x9012FD65
Result: 0xE4E43C50

Memory[28] — DIV (Forward MEM/WB → EX)

Instruction: div r19, r7, r17
Operands: r7 = 0x9012FD65, r17 = 0x9FC59375
Operation: r7 < r17 → quotient 0
Result: 0x00000000
Explanation: r17 forwarded to divider input.

The following is the waveform for this:-



Conclusion:
All results are correct with no pipeline hazards.

Memory Locations 29-30 : Data Hazards

Memory[29] — Load-Use Data Hazard (Stall Insertion)

The instruction sequence for this result is:

lw r11, mem[36]

add r12, r11, r2

The add instruction immediately depends on the value loaded into r11. In a 5-stage pipeline, the lw instruction does not produce its data until the MEM stage, while the add instruction requires the operand during its EX stage one cycle earlier. Therefore, the value of r11 is not yet available in the register file when the add instruction attempts to read it. Forwarding alone cannot resolve this hazard because the memory data has not been generated yet.

The hazard detection unit detects that the destination register of the lw instruction matches a source register of the following add instruction. It responds by stalling the pipeline for one clock cycle: the PC and IF/ID register are frozen and a bubble (NOP) is inserted into the ID/EX stage(at 1070 ns). This delay allows the loaded value to reach the MEM/WB stage, from where it can be forwarded to the ALU input in the next cycle. After the stall, the add instruction receives the correct value and computes:

r12=r11+r2=0xC187A606+0x0FD76E10=0xD15F1416r12 = r11 + r2 = 0xC187A606 + 0x0FD76E10 = 0xD15F1416r12=r11+r2=0xC187A606+0x0FD76E10=0xD15F1416

This value is then stored in memory location 29.

Memory[30] — Second Load-Use Data Hazard

The next instruction pair creates another load-use dependency:

lw r13, mem[32]

add r14, r1, r13

Again, the add instruction requires the value of r13 immediately after it is loaded from memory. Since the data produced by the lw instruction becomes available only in the MEM stage, the following instruction would otherwise read an outdated value from the register file. The hazard detection unit therefore inserts a one-cycle stall identical to the previous case by freezing the PC and IF/ID pipeline registers and inserting a bubble into the execution stage. (at 1130 ns)

After this stall cycle, the loaded value is available and can be forwarded to the ALU. The

processor then performs the addition correctly:

The processor executes:

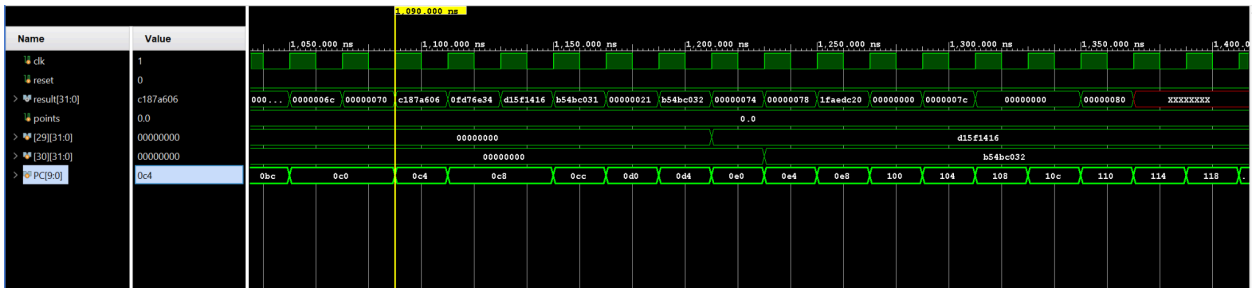
r14=r1+r13

Substitute values:

r14=0x00000001+0xB54BC031

r14 = 0xB54BC032

Then saved in mem[30]



Conclusion:
All results are correct with no pipeline hazards.

Memory Locations 31- : Control Branch Hazard

Instruction sequence:

rom[52] : beq r2, r3, 3

rom[53] : add r10, r1, r2 (wrong-path)

rom[54] : add r10, r1, r4 (wrong-path)

rom[55] : add r10, r1, r5 (wrong-path)

rom[56] : sw r10, 31(r0) (branch target)

Step 1 — Branch condition

From earlier loads:

- r2 = 0x0FD76E10
- r3 = 0x0FD76E10

Since r2 == r3, the branch is taken.

Step 2 — Target address calculation

MIPS branch target uses PC+4:

Target=PC+4+(imm<<2)Target = PC + 4 + (imm << 2)Target=PC+4+(imm<<2)

The BEQ is at rom[52], so:

PC=52×4=0xD0,

PC + 4 + (3<<2) = 0xE0, which is what we get on our waveform. So the processor skips the three sequential instructions at rom[53–55].

Step 3 — Control hazard handling (flush)

Because the pipeline has already fetched at least one instruction after the branch, the processor must prevent wrong-path instructions from executing. When the branch is resolved in the Decode stage, the control unit:

1. redirects the PC to the branch target (0xE0)
2. asserts IF_FLUSH, clearing the IF/ID pipeline register (inserting a NOP). We can see that IF_Flush goes high on the waveform, and q = 00000000, so the pipeline register is also cleared. So, the previous instructions that are being fetched are disregarded.

This ensures rom[53–55] do not reach EX/WB and therefore do not update r10.

Step 4 — Why mem[31] is this value

Since rom[53–55] are flushed, r10 retains its original loaded value:

- r10 = 0xC187A606

Then rom[56] stores it:

mem[31]=r10=0xC187A606 = mem[31]

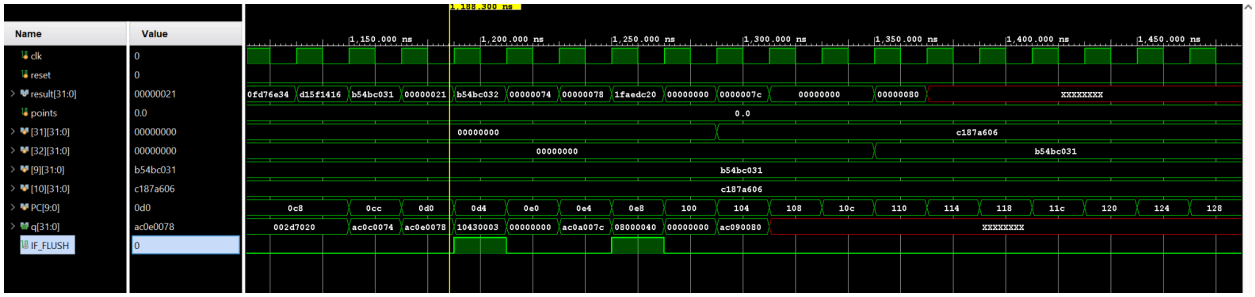
Verification

The testbench/waveform shows:

mem[31] = c187a606

confirming correct branch control hazard handling.

The following is the waveform for this:-



Conclusion:
All results are correct with no pipeline hazards.

Memory Locations 32- : Control Jump Hazard

Instruction sequence:

rom[57] : j 64

rom[58]–rom[63] : add r9, ... (wrong-path)

rom[64] : sw r9, 32(r0) (jump target)

Step 1 — Jump target calculation

A jump is absolute to the encoded target index. Here the target is instruction 64, so:

TargetPC=64×4=0x100TargetPC = 64 \times 4 = 0x100 TargetPC=64×4=0x100

Step 2 — Control hazard handling (flush)

When the jump is decoded, the processor immediately:

1. redirects PC to 0x100
2. asserts IF_FLUSH to clear the IF/ID pipeline register (insert NOP). We can see that IF_Flush goes high on the waveform, and q = 00000000, so the pipeline register is also cleared. So, the previous instructions that are being fetched are disregarded.

This flush prevents the sequentially fetched instructions rom[58–63] from executing, so r9 is not overwritten by any of those adds.

Step 3 — Why mem[32] is this value

Because rom[58–63] are flushed, r9 remains its original loaded value:

- r9 = 0xB54BC031

Then rom[64] stores it:

mem[32]=r9=0xB54BC031 = mem[32]

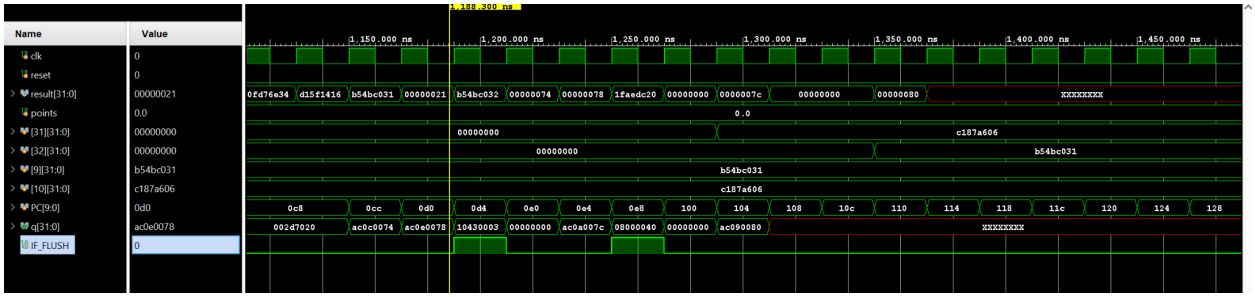
Verification

The testbench/waveform shows:

mem[32] = b54bc031

confirming correct jump control hazard handling.

The following is the waveform for this:-



Conclusion:
All results are correct with no pipeline hazards.